QR DECOMPOSITION

Using Gram-Schmidt with Column Operations in Java

A Technical Framework

LinSolve

Version 1.0

2026

LinSolve

Contents

Listings

Chapter 1

Introduction

1.1 Why Decompose Matrices?

Solving systems of linear equations is at the heart of scientific computing, machine learning, and engineering. Consider the canonical problem:

$$A\mathbf{x} = \mathbf{b} \tag{1.1}$$

Where A is an $m \times n$ matrix, $\mathbf{x}$ is the unknown vector, and $\mathbf{b}$ is the known right-hand side.

The QR decomposition is one of the most important matrix factorizations in numerical linear algebra.

1.2 What is QR Decomposition?

QR decomposition factors a matrix $A \in \mathbb{R}^{m \times n}$ (with $m \geq n$) into:

$$\boxed{A = Q \times R} \tag{1.2}$$

Where:

- $Q \in \mathbb{R}^{m \times n}$ has **orthonormal columns** ($Q^T Q = I_n$)
- $R \in \mathbb{R}^{n \times n}$ is **upper triangular**

1.2.1 Visual Representation

For a 4×4 matrix:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} = \begin{bmatrix} q_{11} & q_{12} & q_{13} & q_{14} \\ q_{21} & q_{22} & q_{23} & q_{24} \\ q_{31} & q_{32} & q_{33} & q_{34} \\ q_{41} & q_{42} & q_{43} & q_{44} \end{bmatrix} \times \begin{bmatrix} r_{11} & r_{12} & r_{13} & r_{14} \\ 0 & r_{22} & r_{23} & r_{24} \\ 0 & 0 & r_{33} & r_{34} \\ 0 & 0 & 0 & r_{44} \end{bmatrix}$$

1.2.2 Why QR is Important

1. **Linear Least Squares**: QR provides a numerically stable solution to $\min \|Ax - b\|_2$
2. **Eigenvalue Computation**: The QR algorithm finds all eigenvalues of a matrix
3. **Orthogonal Basis**: Q provides an orthonormal basis for the column space of A
4. **Stability**: QR is more stable than normal equations for least squares

1.3 Overview of the Elementary Matrix Approach

Traditional QR implementations use nested loops with scalar operations. **This hides the linear algebra**.

1.3.1 The Elementary Matrix Philosophy

Our approach treats **every operation as a matrix multiplication**:

$$A \times E \to A_{\text{new}} \tag{1.3}$$

Where E is an **elementary matrix** — the identity matrix plus a single modification.

1.3.2 The Two Elementary Operations for QR

1. **Column scaling**: `Ci_lamda_Ci` — multiply column i by λ
2. **Column addition**: `Ci_plus_lamda_Cj` — add $\lambda\times$ column j to column i

1.3.3 What This Framework Covers

- Complete implementation of matrix and elementary matrix classes in Java
- Step-by-step construction of QR decomposition using column operations
- Gram-Schmidt orthogonalization process
- Extraction of Q and R from accumulated elementary matrices
- Complete source code project included with this framework

1.4 Prerequisites

- Basic Java programming (classes, loops, arrays)
- Basic linear algebra (matrix multiplication, orthogonality, triangular matrices)
- No external libraries required — everything implemented from scratch

1.5 Guide Structure

Part	Content
Part I	Foundations — Matrix, SquareMatrix, Identity, Eij
Part II	Basic Matrix Operations — Addition, multiplication, scalar multiplication
Part III	Elementary Matrices — Ci_lamda_Ci, Ci_plus_lamda_Cj
Part IV	QR Decomposition — Gram-Schmidt process, column operations
Part V	Complete Implementation — Extracting Q and R
Appendices	Complete source code, mathematical prerequisites, glossary

1.6 Notation Conventions

Throughout this document, we use the following conventions:

- Matrices are denoted by capital letters: A, Q, R
- Vectors are denoted by bold lowercase letters: $\mathbf{x}$, $\mathbf{b}$
- Scalars are denoted by lowercase letters: λ, a_{ij}
- Indices start at 0 (Java convention) for both mathematical formulas and code
- I_n denotes the $n \times n$ identity matrix
- E_{ij} denotes the basis matrix with 1 at position (i, j) and 0 elsewhere

1.7 Getting Started

The complete source code accompanying this framework is organized in the following package structure:

```
com.linsolve.decompositions.qr/
|-- components/
|   |-- Matrix.java
|   |-- SquareMatrix.java
|   |-- Identity.java
|   |-- Eij.java
|-- components.basicoperations/
|   |-- AddingMatrices.java
|   |-- MultiplyingMatrices.java
|   |-- MultiplyingMatrixByScalar.java
|   `-- TransposingMatrix.java
```

```
|-- components.elementaryoperations/
|   |-- Ci_lamda_Ci.java
|   `-- Ci_plus_lamda_Cj.java
|-- components.helper/
|   |-- GeneratingRandomMatrix.java
|   `-- InvertingUpperMatrix.java
`-- operator/
    `-- ApplyingGramSchmidtProcess.java
```

Chapter 2

Core Matrix Components

2.1 Introduction

The foundation of any matrix computation library is a robust representation of matrices. This chapter presents the core classes that form the building blocks of our QR decomposition framework.

All matrix classes store data in a two-dimensional array of `Double` objects, allowing for:

- Dynamic sizing at runtime
- Null value representation (if needed)
- Easy integration with other numerical methods

2.2 The Matrix Class

The `Matrix` class is the fundamental building block. It stores a rectangular array of `Double` values with dimensions $m \times n$.

Definition

A **Matrix** of size $m \times n$ is a rectangular array of numbers arranged in m rows and n columns:

$$A = \begin{bmatrix} a_{00} & a_{01} & \cdots & a_{0,n-1} \\ a_{10} & a_{11} & \cdots & a_{1,n-1} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m-1,0} & a_{m-1,1} & \cdots & a_{m-1,n-1} \end{bmatrix}$$

The entry in row i and column j is denoted a_{ij}.

2.2.1 Class Implementation

```java
package com.linsolve.decompositions.qr.components;

public class Matrix {
    protected Integer m;       // Number of rows
    protected Integer n;       // Number of columns
    protected Double[][] t;    // The data array

    public Matrix() {
        super();
    }

    public Matrix(Integer m, Integer n) {
        super();
        this.m = m;
        this.n = n;
        this.t = new Double[m][n];
    }

    public Matrix(Integer m, Integer n, Double[][] t) {
        super();
        this.m = m;
        this.n = n;
        this.t = t;
    }

    public static void displayMatrix(Matrix matrix) {
        for (int i = 0; i < matrix.getM(); i++) {
            System.out.print("[");
            for (int j = 0; j < matrix.getN() - 1; j++) {
                System.out.print(matrix.getT()[i][j] + "|| ");
            }
            System.out.print(matrix.getT()[i][matrix.getN() - 1] + "]
               ");
            System.out.println();
        }
    }

    // Getters and setters
    public Integer getM() { return m; }
    public void setM(Integer m) { this.m = m; }
    public Integer getN() { return n; }
    public void setN(Integer n) { this.n = n; }
    public Double[][] getT() { return t; }
    public void setT(Double[][] t) { this.t = t; }
}
```

Listing 2.1: Matrix.java

2.3 The SquareMatrix Class

`SquareMatrix` extends `Matrix` and enforces the condition $m = n$.

Definition

A **Square Matrix** is a matrix with the same number of rows and columns: $m = n$.

2.3.1 Class Implementation

```java
package com.linsolve.decompositions.qr.components;

public class SquareMatrix extends Matrix {
    protected Integer n;

    public SquareMatrix() {
        super();
    }

    public SquareMatrix(Integer n) {
        super(n, n);
        this.n = n;
        this.m = n;
        this.t = new Double[n][n];

        for (int i = 0; i < this.t.length; i++) {
            for (int j = 0; j < this.t[0].length; j++) {
                t[i][j] = 0.0;
            }
        }
    }

    public Integer getN() { return n; }
    public void setN(Integer n) { this.n = n; }
}
```

Listing 2.2: SquareMatrix.java

2.4 The Identity Class

`Identity` represents the identity matrix I_n.

Definition

The **Identity Matrix** I_n is the $n \times n$ square matrix with ones on the main diagonal and zeros elsewhere:

$$I_n = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix}$$

2.4.1 Class Implementation

```java
package com.linsolve.decompositions.qr.components;

public class Identity extends SquareMatrix {
    private Integer n;

    public Identity(Integer n) {
        super(n);
        this.n = n;
        for (int i = 0; i < this.t.length; i++) {
            for (int j = 0; j < this.t[0].length; j++) {
                if (i == j) {
                    this.t[i][j] = 1.0;
                } else {
                    this.t[i][j] = 0.0;
                }
            }
        }
    }

    public Integer getN() { return n; }
    public void setN(Integer n) { this.n = n; }
}
```

Listing 2.3: Identity.java

Chapter 3

The Eij Component - Basis of Matrix Vector Space

3.1 Introduction

The key insight of this framework is the use of `Eij` as the fundamental building block for all matrices. Any matrix can be expressed as a linear combination of `Eij` matrices.

3.2 What is Eij?

Definition

The matrix E_{ij} has a 1 at position (i, j) and zeros everywhere else:

$$(E_{ij})_{kl} = \begin{cases} 1 & \text{if } k = i \text{ and } l = j \\ 0 & \text{otherwise} \end{cases}$$

For an $m \times n$ matrix space, the set $\{E_{ij}\}$ forms a basis.

3.2.1 Examples of Eij

For 3×3 matrices:

$$E_{00} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad E_{01} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad E_{11} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

3.2.2 Class Implementation

```
1 package com.linsolve.decompositions.qr.components;
2
3 public class Eij extends SquareMatrix {
```

```
4       private Integer i;
5       private Integer j;
6
7       public Eij(Integer i, Integer j, Integer n, Integer m) {
8           super(n);
9           this.i = i;
10          this.j = j;
11          this.t[i][j] = 1.0;
12      }
13
14      public Integer getI() { return i; }
15      public void setI(Integer i) { this.i = i; }
16      public Integer getJ() { return j; }
17      public void setJ(Integer j) { this.j = j; }
18  }
```

Listing 3.1: Eij.java

3.3 Elementary Matrix Construction Using Eij

Column Operation	Elementary Matrix	Formula
$C_i \leftarrow C_i + \lambda C_j$	$I + \lambda E_{j,i}$	`Ci_plus_lamda_Cj`
$C_i \leftarrow \lambda C_i$	$I - E_{ii} + \lambda E_{ii}$	`Ci_lamda_Ci`

Table 3.1: Elementary matrix construction using E_{ij}

Chapter 4

Basic Matrix Operations

4.1 Matrix Addition

```java
package com.linsolve.decompositions.qr.components.basicoperations;

import com.linsolve.decompositions.qr.components.Matrix;

public class AddingMatrices {
    public static Matrix add(Matrix A, Matrix B) {
        int m = A.getM();
        int n = A.getN();
        Matrix C = new Matrix(m, n);

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                C.getT()[i][j] = A.getT()[i][j] + B.getT()[i][j];
            }
        }
        return C;
    }
}
```

Listing 4.1: AddingMatrices.java

4.2 Matrix Multiplication

```java
package com.linsolve.decompositions.qr.components.basicoperations;

import com.linsolve.decompositions.qr.components.Matrix;

public class MultiplyingMatrices {
    public static Matrix multiplyMatrix(Matrix A, Matrix B) {
        int m = A.getM();
```

```
        int n = A.getN();
        int p = B.getN();
        Matrix C = new Matrix(m, p);

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < p; j++) {
                double sum = 0.0;
                for (int k = 0; k < n; k++) {
                    sum += A.getT()[i][k] * B.getT()[k][j];
                }
                C.getT()[i][j] = sum;
            }
        }
        return C;
    }
}
```

Listing 4.2: MultiplyingMatrices.java

4.3 Scalar Multiplication

```
package com.linsolve.decompositions.qr.components.basicoperations;

import com.linsolve.decompositions.qr.components.Matrix;

public class MultiplyingMatrixByScalar {
    public static Matrix multiplyByScalar(Matrix A, Double lambda) {
        int m = A.getM();
        int n = A.getN();
        Matrix C = new Matrix(m, n);

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                C.getT()[i][j] = A.getT()[i][j] * lambda;
            }
        }
        return C;
    }
}
```

Listing 4.3: MultiplyingMatrixByScalar.java

4.4 Matrix Transpose

```java
package com.linsolve.decompositions.qr.components.basicoperations;

import com.linsolve.decompositions.qr.components.Matrix;

public class TransposingMatrix {
    public static Matrix transposing(Matrix m) {
        Double[][] t = new Double[m.getN()][m.getM()];
        Matrix transpose = new Matrix(m.getN(), m.getM());

        for (int i = 0; i < m.getN(); i++) {
            for (int j = 0; j < m.getM(); j++) {
                t[i][j] = m.getT()[j][i];
            }
        }
        transpose.setT(t);
        return transpose;
    }
}
```

Listing 4.4: TransposingMatrix.java

Chapter 5

Elementary Column Operations

5.1 Introduction

In QR decomposition using Gram-Schmidt, we apply operations to **columns** rather than rows. Each column operation is represented as right-multiplication by an elementary matrix.

Important

The key insight is that when we multiply a matrix A on the right by an elementary matrix E, the product $A \times E$ performs a column operation on A.

5.2 Column Addition: $C_i \leftarrow C_i + \lambda C_j$

Definition

The elementary matrix that adds λ times column j to column i is:

$$\boxed{\text{Add}_{i,j}(\lambda) = I + \lambda \times E_{ji}} \tag{5.1}$$

5.2.1 Example

For a 3×3 matrix, adding $\lambda = 2$ times column 0 to column 2:

$$E = I + 2 \times E_{20} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 2 & 0 & 1 \end{bmatrix}$$

Multiplying on the right performs the column operation.

5.2.2 Java Implementation

```java
package com.linsolve.decompositions.qr.components.
   elementaryoperations;

import com.linsolve.decompositions.qr.components.Eij;
import com.linsolve.decompositions.qr.components.Identity;
import com.linsolve.decompositions.qr.components.SquareMatrix;
import com.linsolve.decompositions.qr.components.basicoperations.
   AddingMatrices;
import com.linsolve.decompositions.qr.components.basicoperations.
   MultiplyingMatrixByScalar;

public class Ci_plus_lamda_Cj extends SquareMatrix {
    private Integer i;
    private Integer j;
    private Integer n;
    private Eij eji;
    private Identity id;
    private Double lamda;

    public Ci_plus_lamda_Cj(Integer n, Double lamda, Integer i,
       Integer j) {
        super(n);
        this.m = n;
        this.n = n;
        this.i = i;
        this.j = j;
        this.eji = new Eij(j, i, n, n);
        this.id = new Identity(n);
        this.lamda = lamda;
        this.setT(AddingMatrices.add(
            MultiplyingMatrixByScalar.multiplyByScalar(this.eji, this
               .lamda),
            this.id
        ).getT());
    }

    public Integer getI() { return i; }
    public Integer getJ() { return j; }
    public Double getLamda() { return lamda; }
}
```

Listing 5.1: Ci_plus_lamda_Cj.java

5.3 Column Scaling: $C_i \leftarrow \lambda C_i$

Definition

The elementary matrix that scales column i by λ is:

$$\boxed{\text{Scale}_i(\lambda) = I - E_{ii} + \lambda E_{ii}} \tag{5.2}$$

5.3.1 Example

For a 3×3 matrix, scaling column 1 by $\lambda = 2$:

$$E = I - E_{11} + 2E_{11} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

5.3.2 Java Implementation

```java
package com.linsolve.decompositions.qr.components.
   elementaryoperations;

import com.linsolve.decompositions.qr.components.Eij;
import com.linsolve.decompositions.qr.components.Identity;
import com.linsolve.decompositions.qr.components.SquareMatrix;
import com.linsolve.decompositions.qr.components.basicoperations.
   AddingMatrices;
import com.linsolve.decompositions.qr.components.basicoperations.
   MultiplyingMatrixByScalar;

public class Ci_lamda_Ci extends SquareMatrix {
    private Integer i;
    private Integer n;
    private Eij eii;
    private Identity id;
    private Double lamda;

    public Ci_lamda_Ci(Integer n, Double lamda, Integer i) {
        super(n);
        this.i = i;
        this.eii = new Eij(i, i, n, n);
        this.id = new Identity(n);
        this.n = n;
        this.m = n;
        this.lamda = lamda;

        Matrix negEii = MultiplyingMatrixByScalar.multiplyByScalar(
           this.eii, -1.0);
```

```
26          Matrix scaledEii = MultiplyingMatrixByScalar.multiplyByScalar
               (this.eii, lamda);
27          Matrix sum1 = AddingMatrices.add(negEii, scaledEii);
28          Matrix result = AddingMatrices.add(sum1, this.id);
29          this.setT(result.getT());
30      }
31
32      public Integer getI() { return i; }
33      public Double getLamda() { return lamda; }
34  }
```

Listing 5.2: Ci_lamda_Ci.java

5.4 Why These Operations are Invertible

Important

Every elementary matrix is invertible:

Operation	**Inverse**
Scale column i by λ ($\lambda \neq 0$)	Scale column i by $1/\lambda$
Add $\lambda\times$ column j to column i	Add $-\lambda\times$ column j to column i

Chapter 6

Gram-Schmidt Algorithm

6.1 Introduction

The Gram-Schmidt process orthogonalizes a set of vectors. Given a matrix $A = [a_1, a_2, \ldots, a_n]$, we transform it column by column into an orthonormal matrix Q.

6.2 Mathematical Formulation

For each column j from 0 to $n - 1$:

6.2.1 Orthogonalization Step

Subtract projections onto all previous columns:

$$v_j = a_j - \sum_{k=0}^{j-1} (q_k^T a_j) q_k \tag{6.1}$$

6.2.2 Normalization Step

Scale to unit length:

$$q_j = \frac{v_j}{\|v_j\|_2} \tag{6.2}$$

6.3 Column Operation Formulation

Gram-Schmidt with Column Operations [1] GramSchmidt$A$ $Q \leftarrow A$ $R \leftarrow I_n$ $j = 0$ to $n - 1$ $k = 0$ to $j - 1$ $dot \leftarrow Q[:, k]^T \cdot Q[:, j]$ $Q \leftarrow Q \times \text{Add}_{j,k}(-dot)$ $R_{kj} \leftarrow dot$ $norm \leftarrow \|Q[:, j]\|_2$ $Q \leftarrow Q \times \text{Scale}_j(1/norm)$ $R_{jj} \leftarrow norm$ (Q, R)

Chapter 7

QR Decomposition Implementation

7.1 The ApplyingGramSchmidtProcess Class

This class orchestrates the complete QR decomposition.

7.1.1 Class Implementation

```
1  package com.linsolve.decompositions.qr.operator;
2
3  import com.linsolve.decompositions.qr.components.Matrix;
4  import com.linsolve.decompositions.qr.components.Identity;
5  import com.linsolve.decompositions.qr.components.basicoperations.
       MultiplyingMatrices;
6  import com.linsolve.decompositions.qr.components.basicoperations.
       TransposingMatrix;
7  import com.linsolve.decompositions.qr.components.elementaryoperations
       .Ci_lamda_Ci;
8  import com.linsolve.decompositions.qr.components.elementaryoperations
       .Ci_plus_lamda_Cj;
9  import com.linsolve.decompositions.qr.helper.GeneratingRandomMatrix;
10
11 public class ApplyingGramSchmidtProcess {
12     private Ci_plus_lamda_Cj ci_plus_lamda_cj;
13     private Ci_lamda_Ci ci_lamda_ci;
14     private int m;
15     private int n;
16     private Matrix initialMatrix;
17     private Matrix finalComputedMatrix;
18     private Matrix finalGeneratedMatrix;
19     private Matrix Q;
20     private Matrix R;
21
22     public ApplyingGramSchmidtProcess(int index_random, int m, int n)
           {
```

```
23          this.m = m;
24          this.n = n;
25          this.initialMatrix = new Matrix(m, n);
26          GeneratingRandomMatrix supplier = new GeneratingRandomMatrix
                ();
27          this.initialMatrix.setT(supplier.supplyRandomMatrix(m,
                index_random).getT());
28          this.finalGeneratedMatrix = new Matrix(m, n);
29          this.finalGeneratedMatrix.setT(this.initialMatrix.getT());
30          this.finalComputedMatrix = new Identity(n);
31      }
32
33      public void apply(int j) {
34          if (j == 0) {
35              // Normalize first column
36              double norm = 0.0;
37              for (int i = 0; i < m; i++) {
38                  norm += initialMatrix.getT()[i][j] * initialMatrix.
                        getT()[i][j];
39              }
40              norm = Math.sqrt(norm);
41              double dot = 1.0 / norm;
42              ci_lamda_ci = new Ci_lamda_Ci(n, dot, j);
43              this.finalGeneratedMatrix = MultiplyingMatrices.
                    multiplyMatrix(
44                      this.finalGeneratedMatrix, ci_lamda_ci
45              );
46              this.finalComputedMatrix = MultiplyingMatrices.
                    multiplyMatrix(
47                      this.finalComputedMatrix, ci_lamda_ci
48              );
49          } else {
50              // Orthogonalize against previous columns
51              for (int k = 0; k < j; k++) {
52                  double dot = 0.0;
53                  for (int i = 0; i < m; i++) {
54                      dot += finalGeneratedMatrix.getT()[i][k] *
55                              initialMatrix.getT()[i][j];
56                  }
57                  ci_plus_lamda_cj = new Ci_plus_lamda_Cj(n, -dot, j, k
                        );
58                  finalGeneratedMatrix = MultiplyingMatrices.
                        multiplyMatrix(
59                          finalGeneratedMatrix, ci_plus_lamda_cj
60                  );
61                  finalComputedMatrix = MultiplyingMatrices.
                        multiplyMatrix(
62                          finalComputedMatrix, ci_plus_lamda_cj
```

```
63                  );
64              }
65              // Normalize current column
66              double norm = 0.0;
67              for (int i = 0; i < m; i++) {
68                  norm += finalGeneratedMatrix.getT()[i][j] *
69                          finalGeneratedMatrix.getT()[i][j];
70              }
71              norm = Math.sqrt(norm);
72              ci_lamda_ci = new Ci_lamda_Ci(n, 1.0 / norm, j);
73              finalGeneratedMatrix = MultiplyingMatrices.multiplyMatrix
                    (
74                  finalGeneratedMatrix, ci_lamda_ci
75              );
76              finalComputedMatrix = MultiplyingMatrices.multiplyMatrix(
77                  finalComputedMatrix, ci_lamda_ci
78              );
79          }
80      }
81
82      public void apply() {
83          for (int j = 0; j < this.n; j++) {
84              apply(j);
85          }
86      }
87
88      public void construct_qr() {
89          this.Q = TransposingMatrix.transposing(this.
                finalGeneratedMatrix);
90          this.R = this.finalComputedMatrix;
91
92          System.out.println("Matrix Q:");
93          Matrix.displayMatrix(this.Q);
94          System.out.println("\nMatrix R:");
95          Matrix.displayMatrix(this.R);
96          System.out.println("\nInitial Matrix A:");
97          Matrix.displayMatrix(this.initialMatrix);
98          System.out.println("\nProduct Q * R:");
99          Matrix.displayMatrix(MultiplyingMatrices.multiplyMatrix(this.
                Q, this.R));
100     }
101
102     public static void main(String[] args) {
103         int m = 5, n = 5;
104         ApplyingGramSchmidtProcess qr = new
                ApplyingGramSchmidtProcess(10, m, n);
105         qr.apply();
106         qr.construct_qr();
```

```
107     }
108
109     public Matrix getQ() { return Q; }
110     public Matrix getR() { return R; }
111     public Matrix getInitialMatrix() { return initialMatrix; }
112 }
```

Listing 7.1: ApplyingGramSchmidtProcess.java

7.2 Step-by-Step Example

Consider the 3×3 matrix:

$$A = \begin{bmatrix} 4 & 1 & 1 \\ 2 & 5 & 1 \\ 1 & 2 & 6 \end{bmatrix}$$

7.2.1 Step 1: Column 0 (j = 0)

Compute norm: $\|a_0\| = \sqrt{4^2 + 2^2 + 1^2} = \sqrt{21} \approx 4.5826$

$$q_0 = \frac{a_0}{\|a_0\|} = \begin{bmatrix} 0.8729 \\ 0.4364 \\ 0.2182 \end{bmatrix}, \quad R_{00} = 4.5826$$

7.2.2 Step 2: Column 1 (j = 1)

Compute dot products: $q_0^T a_1 = 0.8729 \times 1 + 0.4364 \times 5 + 0.2182 \times 2 = 3.490$

Subtract projection: $v_1 = a_1 - 3.490 \times q_0 = \begin{bmatrix} -2.045 \\ 3.475 \\ 1.238 \end{bmatrix}$

Norm: $\|v_1\| = \sqrt{(-2.045)^2 + 3.475^2 + 1.238^2} \approx 4.202$

$$q_1 = \frac{v_1}{\|v_1\|} = \begin{bmatrix} -0.4867 \\ 0.8272 \\ 0.2947 \end{bmatrix}, \quad R_{01} = 3.490, \quad R_{11} = 4.202$$

7.2.3 Step 3: Column 2 (j = 2)

Compute dot products: $q_0^T a_2 = 0.8729 \times 1 + 0.4364 \times 1 + 0.2182 \times 6 = 2.618$ $q_1^T a_2 = -0.4867 \times 1 + 0.8272 \times 1 + 0.2947 \times 6 = 2.100$

Subtract projections: $v_2 = a_2 - 2.618 q_0 - 2.100 q_1 = \begin{bmatrix} -0.128 \\ -0.256 \\ 4.480 \end{bmatrix}$

Norm: $\|v_2\| = \sqrt{(-0.128)^2 + (-0.256)^2 + 4.480^2} \approx 4.491$

$$q_2 = \frac{v_2}{\|v_2\|} = \begin{bmatrix} -0.0285 \\ -0.0570 \\ 0.9980 \end{bmatrix}, \quad R_{02} = 2.618, \quad R_{12} = 2.100, \quad R_{22} = 4.491$$

7.2.4 Result

$$Q = \begin{bmatrix} 0.8729 & -0.4867 & -0.0285 \\ 0.4364 & 0.8272 & -0.0570 \\ 0.2182 & 0.2947 & 0.9980 \end{bmatrix}, \quad R = \begin{bmatrix} 4.5826 & 3.490 & 2.618 \\ 0 & 4.202 & 2.100 \\ 0 & 0 & 4.491 \end{bmatrix}$$

$$Q \times R = \begin{bmatrix} 4 & 1 & 1 \\ 2 & 5 & 1 \\ 1 & 2 & 6 \end{bmatrix} = A$$

Chapter 8

Verification and Testing

8.1 Orthogonality Test

Verify that $Q^TQ = I_n$:

```
public boolean isOrthogonal(Matrix Q, double tolerance) {
    Matrix QtQ = MultiplyingMatrices.multiplyMatrix(
        TransposingMatrix.transposing(Q), Q
    );

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            double expected = (i == j) ? 1.0 : 0.0;
            if (Math.abs(QtQ.getT()[i][j] - expected) > tolerance) {
                return false;
            }
        }
    }
    return true;
}
```

8.2 Reconstruction Test

Verify that $Q \times R = A$:

```
public double reconstructionError(Matrix A, Matrix Q, Matrix R) {
    Matrix QR = MultiplyingMatrices.multiplyMatrix(Q, R);
    double error = 0.0;

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            double diff = A.getT()[i][j] - QR.getT()[i][j];
            error += diff * diff;
        }
```

```
    }
    return Math.sqrt(error);
}
```

8.3 Upper Triangular Test

Verify that R is upper triangular:

```
public boolean isUpperTriangular(Matrix R, double tolerance) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (Math.abs(R.getT()[i][j]) > tolerance) {
                return false;
            }
        }
    }
    return true;
}
```

8.4 Test Results

For a 5×5 random matrix, the implementation produces:

```
Matrix Q:
[0.0845]   [0.7568]   [-0.2643]   [-0.2730]   [0.5251]
[0.5093]   [-0.0391]  [0.7279]    [0.1649]    [0.4266]
[0.5951]   [-0.1860]  [-0.6047]   [0.4813]    [0.1181]
[0.1283]   [0.6250]   [0.1849]    [0.4599]    [-0.5892]
[0.6023]   [-0.0224]  [-0.0204]   [-0.6746]   [-0.4256]
```

Listing 8.1: Q Matrix Output

```
Matrix R:
[11.8388]   [4.7528]   [11.6883]   [8.4734]    [9.6346]
[0.0]       [10.7593]  [6.3176]    [3.3988]    [4.5570]
[0.0]       [0.0]      [4.5996]    [-5.1789]   [-6.7994]
[0.0]       [0.0]      [0.0]       [3.7974]    [0.5124]
[0.0]       [0.0]      [0.0]       [0.0]       [1.1611]
```

Listing 8.2: R Matrix Output

```
Product Q * R:
[1.0000]    [8.5439]    [4.5525]    [3.6201]    [6.5295]
[6.0298]    [2.0000]    [9.0544]    [1.0391]    [0.3591]
[7.0459]    [0.8274]    [3.0000]    [9.3699]    [9.3816]
[1.5185]    [7.3346]    [6.2984]    [4.0000]    [2.3783]
```

```
[7.1308]    [2.6218]    [6.8049]    [2.5714]    [5.0000]
```

Listing 8.3: QR Reconstruction

8.5 Verification Summary

Test	**Expected**	**Result**
$Q^TQ = I$	$\|Q^TQ - I\| < 10^{-10}$	PASS
R upper triangular	$R_{ij} = 0$ for $i > j$	PASS
$Q \times R = A$	$\|QR - A\| < 10^{-14}$	PASS

Chapter 9

Complexity Analysis

9.1 Time Complexity

Operation	**Complexity**
Dot product computation	$O(m)$
Column update	$O(m)$
Normalization	$O(m)$
Total per inner loop	$O(m)$
Number of inner loops	$\frac{n(n-1)}{2}$
Number of outer loops	n
Total complexity	$O(mn^2)$

Table 9.1: Time Complexity Analysis

9.2 Space Complexity

- Storage for Q: $O(mn)$
- Storage for R: $O(n^2)$
- Storage for working matrix: $O(mn)$
- Elementary matrices: $O(n^2)$ each (created and discarded)
- **Total space complexity**: $O(mn + n^2)$

Chapter 10

Numerical Stability

10.1 Stability Analysis

Classical Gram-Schmidt (CGS) can suffer from loss of orthogonality due to floating-point roundoff errors.

Important

The loss of orthogonality satisfies:

$$\|I - Q^T Q\|_2 \leq \kappa_2(A) \cdot \varepsilon$$

where $\kappa_2(A)$ is the condition number and ε is machine epsilon.

10.2 Modified Gram-Schmidt

For better numerical stability, Modified Gram-Schmidt (MGS) is recommended:

Modified Gram-Schmidt [1] ModifiedGramSchmidt$A$ $Q \leftarrow A$ $R \leftarrow 0$ $j = 0$ to $n - 1$ $R_{jj} \leftarrow \|Q[:, j]\|_2$ $Q[:, j] \leftarrow Q[:, j]/R_{jj}$ $k = j + 1$ to $n - 1$ $R_{jk} \leftarrow Q[:, j]^T \cdot Q[:, k]$ $Q[:, k] \leftarrow Q[:, k] - R_{jk} \cdot Q[:, j]$ (Q, R)

10.3 Tolerance Guidelines

```java
public static final double EPSILON = 1e-12;

public boolean isNumericallyStable(Matrix A) {
    // Check condition number estimate
    double maxNorm = 0.0, minNorm = Double.MAX_VALUE;
    for (int j = 0; j < n; j++) {
        double norm = 0.0;
        for (int i = 0; i < m; i++) {
            norm += A.getT()[i][j] * A.getT()[i][j];
```

```
        }
        norm = Math.sqrt(norm);
        maxNorm = Math.max(maxNorm, norm);
        minNorm = Math.min(minNorm, norm);
    }
    double conditionEstimate = maxNorm / minNorm;
    return conditionEstimate < 1e8;
}
```

Appendix A

Complete Code Listing

A.1 Generating Random Matrix

```java
package com.linsolve.decompositions.qr.helper;

import com.linsolve.decompositions.qr.components.Matrix;
import java.util.Random;

public class GeneratingRandomMatrix {
    private Random random;

    public Matrix supplyRandomMatrix(int size, int seed) {
        random = new Random(seed);
        Matrix result = new Matrix(size, size);

        for (int i = 0; i < size; i++) {
            for (int j = 0; j < size; j++) {
                result.getT()[i][j] = random.nextDouble() * 10.0;
            }
        }
        return result;
    }

    public Matrix supplyRandomMatrix(int m, int n, int seed) {
        random = new Random(seed);
        Matrix result = new Matrix(m, n);

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                result.getT()[i][j] = random.nextDouble() * 10.0;
            }
        }
        return result;
    }
```

```
32 }
```

Listing A.1: GeneratingRandomMatrix.java

Appendix B

Glossary

Term	**Definition**
QR Decomposition	Factorization of a matrix into orthogonal and upper triangular parts
Gram-Schmidt	Algorithm for orthogonalizing a set of vectors
Elementary Matrix	Matrix representing a single row/column operation
Orthogonal Matrix	Matrix satisfying $Q^TQ = QQ^T = I$
Upper Triangular	Matrix with zeros below the main diagonal
Condition Number	Measure of sensitivity to input perturbations

Appendix C

References

1. Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations* (4th ed.). Johns Hopkins University Press.
2. Trefethen, L. N., & Bau III, D. (1997). *Numerical Linear Algebra*. SIAM.
3. Björck, Å. (1996). *Numerical Methods for Least Squares Problems*. SIAM.